

Sistema de Gestão de Condomínios

Sistemas de Informação | Desenvolvimento Full Stack | Semestre 01/2026

(Refatoração, Design Dark Mode e Boas Práticas)

PostgreSQL — Banco de Dados

NestJS — Backend

Next.js — Frontend

shadcn/ui — Componentes

JWT — Autenticação

Sumário

Parte	Título	Conteúdo
Parte 1	Modelo de Dados e Banco de Dados	Revisão e melhoria do script SQL
Parte 2	Backend com NestJS — Refatoração Completa	Estrutura, entidades, CRUD
Parte 3	Autenticação JWT — Implementação Detalhada	Login seguro, hash de senha, guarda de rotas
Parte 4	Frontend com Next.js — Refatoração e Design	Instalação do shadcn/ui, layout dark mode
Parte 5	Tela de Login — Dark Mode com shadcn/ui	Formulário seguro e estilizado
Parte 6	Dashboard Moderno	Cards de métricas, sidebar, navegação
Parte 7	Páginas CRUD Refatoradas	Tabelas, formulários e feedback visual
Parte 8	Integração Final e Testes	Fluxo completo, Postman, checklist

Parte 1 — Modelo de Dados e Banco de Dados

 **Refatoração** O script SQL original estava correto para PostgreSQL, mas o backend da Parte 2 utiliza MySQL. Esta seção apresenta o script revisado para MySQL, compatível com o TypeORM configurado no NestJS.

1.1 Pré-requisitos

- MySQL 8.x instalado e em execução
- MySQL Workbench (opcional, mas recomendado)
- Acesso de root ao servidor MySQL

1.2 Criando o Banco de Dados

Abra o MySQL Workbench ou o terminal e execute:

```
SQL — criar banco
-- Criar e selecionar o banco
CREATE DATABASE IF NOT EXISTS DBCONDOMINIO
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE DBCONDOMINIO;
```

1.3 Script SQL Completo (MySQL — Revisado)

 **Atenção — Ordem de criação** Tabelas com chaves estrangeiras devem ser criadas APÓS as tabelas que referenciam. A ordem abaixo já está correta.

```
SQL — PESSOAS
-- =====
-- TABELA PESSOAS (base de todos os cadastros)
-- =====
CREATE TABLE PESSOAS (
  ID_PESSOA      INT AUTO_INCREMENT PRIMARY KEY,
  NOME           VARCHAR(150) NOT NULL,
  TIPO_PESSOA    ENUM('FISICA', 'JURIDICA') NOT NULL,
  CPF_CNPJ       VARCHAR(20) NOT NULL UNIQUE,
  DATA_CADASTRO DATE          NOT NULL
);
```

```
SQL — CONTATOS e ENDEREÇOS
CREATE TABLE CONTATOS (
  ID_CONTATO     INT AUTO_INCREMENT PRIMARY KEY,
  ID_PESSOA      INT NOT NULL,
  TIPO_CONTATO   ENUM('TELEFONE', 'EMAIL', 'WHATSAPP') NOT NULL,
  VALOR_CONTATO  VARCHAR(100) NOT NULL,
```

```

        FOREIGN KEY (ID_PESSOA) REFERENCES PESSOAS(ID_PESSOA)
        ON DELETE CASCADE
    );

CREATE TABLE ENDEREÇOS (
    ID_ENDERECO    INT AUTO_INCREMENT PRIMARY KEY,
    ID_PESSOA      INT NOT NULL,
    LOGRADOURO     VARCHAR(150),
    NUMERO         VARCHAR(10),
    BAIRRO         VARCHAR(100),
    CIDADE         VARCHAR(100),
    UF             CHAR(2),
    CEP            VARCHAR(9),
    FOREIGN KEY (ID_PESSOA) REFERENCES PESSOAS(ID_PESSOA)
    ON DELETE CASCADE
);

```

SQL – UNIDADES, MORADORES, FUNCIONARIOS, FORNECEDORES, VISITANTES

```

CREATE TABLE UNIDADES (
    ID_UNIDADE     INT AUTO_INCREMENT PRIMARY KEY,
    NUM_UNIDADE    VARCHAR(10) NOT NULL,
    BLOCO          VARCHAR(10) NOT NULL,
    TIPO           VARCHAR(20),
    AREA_TOTAL     DECIMAL(10,2)
);

CREATE TABLE MORADORES (
    ID_MORADOR     INT AUTO_INCREMENT PRIMARY KEY,
    ID_PESSOA      INT NOT NULL,
    ID_UNIDADE     INT NOT NULL,
    FOREIGN KEY (ID_PESSOA) REFERENCES PESSOAS(ID_PESSOA),
    FOREIGN KEY (ID_UNIDADE) REFERENCES UNIDADES(ID_UNIDADE)
);

CREATE TABLE FUNCIONARIOS (
    ID_FUNCIONARIO INT AUTO_INCREMENT PRIMARY KEY,
    ID_PESSOA      INT NOT NULL,
    FUNCAO         VARCHAR(50),
    DATA_ADMISSAO DATE,
    SALARIO        DECIMAL(10,2),
    FOREIGN KEY (ID_PESSOA) REFERENCES PESSOAS(ID_PESSOA)
);

CREATE TABLE FORNECEDORES (
    ID_FORNECEDOR  INT AUTO_INCREMENT PRIMARY KEY,
    ID_PESSOA      INT NOT NULL,
    AREA_ATUACAO   VARCHAR(100),
    FOREIGN KEY (ID_PESSOA) REFERENCES PESSOAS(ID_PESSOA)
);

CREATE TABLE VISITANTES (
    ID_VISITANTE   INT AUTO_INCREMENT PRIMARY KEY,
    ID_PESSOA      INT NOT NULL,

```

```

        DOCUMENTO      VARCHAR(20),
        FOREIGN KEY (ID_PESSOA) REFERENCES PESSOAS(ID_PESSOA)
    );

```

SQL – AREAS_COMUNS, RESERVAS, BOLETOS

```

CREATE TABLE AREAS_COMUNS (
    ID_AREA_COMUM INT AUTO_INCREMENT PRIMARY KEY,
    NOME_AREA      VARCHAR(50) NOT NULL,
    DESCR_AREA     TEXT,
    CAPACIDADE     INT
);

CREATE TABLE RESERVAS (
    ID_RESERVA      INT AUTO_INCREMENT PRIMARY KEY,
    ID_AREA_COMUM   INT NOT NULL,
    ID_MORADOR      INT NOT NULL,
    DATA_RESERVA   DATE NOT NULL,
    HR_INICIO       TIME,
    HR_FIM          TIME,
    FOREIGN KEY (ID_AREA_COMUM) REFERENCES AREAS_COMUNS(ID_AREA_COMUM),
    FOREIGN KEY (ID_MORADOR)   REFERENCES MORADORES(ID_MORADOR)
);

CREATE TABLE BOLETOS (
    ID_BOLETO       INT AUTO_INCREMENT PRIMARY KEY,
    ID_MORADOR      INT NOT NULL,
    VL_BOLETO       DECIMAL(10,2) NOT NULL,
    DT_VENCIMENTO   DATE NOT NULL,
    STATUS          ENUM('ABERTO','PAGO','ATRASADO') DEFAULT 'ABERTO',
    FOREIGN KEY (ID_MORADOR) REFERENCES MORADORES(ID_MORADOR)
);

```

SQL – COMUNICADOS, CONTRATOS, CONTRATOS_RH

```

CREATE TABLE COMUNICADOS (
    ID_COMUNICADO INT AUTO_INCREMENT PRIMARY KEY,
    TITULO         VARCHAR(100) NOT NULL,
    MENSAGEM       TEXT NOT NULL,
    DT_COMUNICADO  DATE NOT NULL,
    HR_COMUNICADO  TIME,
    TIPO           VARCHAR(30)
);

CREATE TABLE CONTRATOS (
    ID_CONTRATO    INT AUTO_INCREMENT PRIMARY KEY,
    ID_FORNECEDOR  INT NOT NULL,
    DESCRICAO      TEXT,
    DATA_INICIO   DATE,
    DATA_FIM      DATE,
    VALOR          DECIMAL(10,2),
    FOREIGN KEY (ID_FORNECEDOR) REFERENCES FORNECEDORES(ID_FORNECEDOR)
);

```

```

CREATE TABLE CONTRATOS_RH (
    ID_CONTRATO_RH    INT AUTO_INCREMENT PRIMARY KEY,
    ID_FUNCIONARIO    INT NOT NULL,
    DESCRICAO         TEXT,
    DATA_INICIO      DATE,
    DATA_FIM         DATE,
    SALARIO_ACORDADO  DECIMAL(10,2),
    FOREIGN KEY (ID_FUNCIONARIO) REFERENCES FUNCIONARIOS(ID_FUNCIONARIO)
);

```

SQL – VISITAS, CONTAS_PAGAR, CONTAS_RECEBER

```

CREATE TABLE VISITAS (
    ID_VISITA          INT AUTO_INCREMENT PRIMARY KEY,
    ID_VISITANTE       INT NOT NULL,
    ID_UNIDADE         INT NOT NULL,
    ID_MORADOR_AUTORIZACAO INT NOT NULL,
    PLACA_VEICULO      VARCHAR(10),
    DATA_ENTRADA      DATETIME,
    DATA_SAIDA        DATETIME,
    FOREIGN KEY (ID_VISITANTE) REFERENCES VISITANTES(ID_VISITANTE),
    FOREIGN KEY (ID_UNIDADE) REFERENCES UNIDADES(ID_UNIDADE),
    FOREIGN KEY (ID_MORADOR_AUTORIZACAO) REFERENCES MORADORES(ID_MORADOR)
);

```

```

CREATE TABLE CONTAS_PAGAR (
    ID_CONTA_PAGAR     INT AUTO_INCREMENT PRIMARY KEY,
    ID_FORNECEDOR      INT NOT NULL,
    DESCRICAO          TEXT,
    VALOR              DECIMAL(10,2),
    DATA_VENCIMENTO   DATE,
    STATUS             ENUM('ABERTO','PAGO','ATRASADO') DEFAULT 'ABERTO',
    FOREIGN KEY (ID_FORNECEDOR) REFERENCES FORNECEDORES(ID_FORNECEDOR)
);

```

```

CREATE TABLE CONTAS_RECEBER (
    ID_CONTA_RECEBER   INT AUTO_INCREMENT PRIMARY KEY,
    ID_MORADOR         INT NOT NULL,
    DESCRICAO          TEXT,
    VALOR              DECIMAL(10,2),
    DATA_VENCIMENTO   DATE,
    STATUS             ENUM('ABERTO','PAGO','ATRASADO') DEFAULT 'ABERTO',
    FOREIGN KEY (ID_MORADOR) REFERENCES MORADORES(ID_MORADOR)
);

```

SQL – PAGAMENTOS, RECEBIMENTOS, CONTA_CORRENTE, MOV_CONTA_CORRENTE

```

CREATE TABLE PAGAMENTOS (
    ID_PAGAMENTO       INT AUTO_INCREMENT PRIMARY KEY,
    ID_CONTA_PAGAR     INT NOT NULL,
    DATA_PAGAMENTO    DATE,
    VALOR_PAGO         DECIMAL(10,2),
    FORMA_PAGAMENTO    VARCHAR(30),
    FOREIGN KEY (ID_CONTA_PAGAR) REFERENCES CONTAS_PAGAR(ID_CONTA_PAGAR)
);

```

```

CREATE TABLE RECEBIMENTOS (
    ID_RECEBIMENTO INT AUTO_INCREMENT PRIMARY KEY,
    ID_CONTA_RECEBER INT NOT NULL,
    DATA_RECEBIMENTO DATE,
    VALOR_RECEBIDO DECIMAL(10,2),
    FORMA_RECEBIMENTO VARCHAR(30),
    FOREIGN KEY (ID_CONTA_RECEBER) REFERENCES CONTAS_RECEBER(ID_CONTA_RECEBER)
);

CREATE TABLE CONTA_CORRENTE (
    ID_CONTA_CORRENTE INT AUTO_INCREMENT PRIMARY KEY,
    BANCO VARCHAR(50),
    AGENCIA VARCHAR(20),
    NUM_CONTA VARCHAR(20),
    SALDO_ATUAL DECIMAL(12,2)
);

CREATE TABLE MOV_CONTA_CORRENTE (
    ID_MOVIMENTO INT AUTO_INCREMENT PRIMARY KEY,
    ID_CONTA_CORRENTE INT NOT NULL,
    ID_CONTA INT,
    ORIGEM_CONTA ENUM('PAGAR', 'RECEBER'),
    TIPO_MOVIMENTO ENUM('CREDITO', 'DEBITO'),
    VALOR DECIMAL(10,2),
    DATA_MOVIMENTO DATE,
    HR_MOVIMENTO TIME,
    DESCRICAO TEXT,
    FOREIGN KEY (ID_CONTA_CORRENTE) REFERENCES CONTA_CORRENTE(ID_CONTA_CORRENTE)
);

```

☒ **Parte 1 concluída** Banco DBCONDOMINIO criado com todas as tabelas, relacionamentos e ENUMs. Guarde o script em um arquivo .sql para controle de versão.

Parte 2 — Backend com NestJS (Refatoração Completa)

💡 **O que é refatoração?** Refatorar significa reescrever ou reorganizar um código existente para torná-lo mais seguro, legível e correto — sem alterar o que ele faz. Aqui vamos corrigir problemas do código original e adicionar boas práticas.

2.1 Pré-requisitos

- Node.js LTS instalado (verifique com: `node -v`)
- NestJS CLI instalado (`npm install -g @nestjs/cli`)
- MySQL rodando com o banco DBCONDOMINIO criado

2.2 Criando o Projeto

Terminal

```
nest new condominio-backend  
cd condominio-backend
```

2.3 Instalando Dependências

Terminal — instalação de pacotes

```
# Banco de dados  
npm install --save @nestjs/typeorm typeorm mysql2  
  
# Validação de dados  
npm install --save class-validator class-transformer  
  
# Autenticação (já instalar para a Parte 3)  
npm install --save @nestjs/jwt @nestjs/passport passport passport-jwt bcryptjs  
npm install --save-dev @types/passport-jwt @types/bcryptjs
```

Dependência de desenvolvimento (--save-dev): usada apenas durante o desenvolvimento, testes ou build. Não é necessária quando a aplicação está rodando em produção.

Dependência normal (--save): usada em tempo de execução da aplicação. Ex.: bibliotecas que o código precisa para funcionar em produção.

2.4 Configuração do Banco de Dados

Edite o arquivo `src/app.module.ts` com o conteúdo abaixo. Substitua `'sua_senha'` pela senha do seu MySQL:

⚠ **Segurança** Nunca suba credenciais de banco de dados para o GitHub. Mais adiante você aprenderá a usar variáveis de ambiente (.env).

src/app.module.ts

```
// src/app.module.ts
import { Module }      from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type:          'mysql',
      host:           'localhost',
      port:           3306,
      username:       'root',
      password:       'sua_senha',
      database:       'DBCONDOMINIO',
      autoLoadEntities: true,
      synchronize:    false, // IMPORTANTE: false em produção
    }),
  ],
})
export class AppModule {}
```

⚠ **synchronize: false** O material original usava synchronize: true, que recria tabelas automaticamente e pode apagar dados. Use sempre false após criar as tabelas manualmente com o script SQL.

2.5 Habilitando ValidationPipe Globalmente

Abra src/main.ts e adicione o ValidationPipe. Isso faz a validação dos dados funcionar automaticamente em todos os endpoints:

src/main.ts

```
// src/main.ts
import { NestFactory }    from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { AppModule }      from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // Habilita validação automática em todos os endpoints
  app.useGlobalPipes(new ValidationPipe({
    whitelist:      true, // remove campos não declarados no DTO
    forbidNonWhitelisted: true,
    transform:      true, // converte tipos automaticamente
  }));

  // Habilita CORS para o frontend Next.js
  app.enableCors({ origin: 'http://localhost:3001' });
}
```

```
    await app.listen(3000);
  }
  bootstrap();
```

2.6 Criando a Entidade e o DTO de Pessoas

No NestJS com boas práticas, separamos a entidade (mapeamento do banco) do DTO (dados que o usuário pode enviar). Isso evita que campos como ID sejam enviados manualmente.

src/pessoas/pessoa.entity.ts

```
// src/pessoas/pessoa.entity.ts
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn }
  from 'typeorm';

@Entity('PESSOAS')
export class Pessoa {
  @PrimaryGeneratedColumn()
  ID_PESSOA: number;

  @Column({ length: 150 })
  NOME: string;

  @Column({ length: 20 })
  TIPO_PESSOA: string; // 'FISICA' ou 'JURIDICA'

  @Column({ unique: true, length: 20 })
  CPF_CNPJ: string;

  @Column({ type: 'date' })
  DATA_CADASTRO: Date;
}
```

src/pessoas/dto/create-pessoa.dto.ts

```
// src/pessoas/dto/create-pessoa.dto.ts
import { IsString, IsNotEmpty, IsDateString,
  IsIn, MaxLength } from 'class-validator';

export class CreatePessoaDto {
  @IsString()
  @IsNotEmpty({ message: 'O nome é obrigatório' })
  @MaxLength(150)
  NOME: string;

  @IsIn(['FISICA', 'JURIDICA'], {
    message: 'Tipo deve ser FISICA ou JURIDICA'
  })
  TIPO_PESSOA: string;

  @IsString()
```

```

    @IsNotEmpty({ message: 'CPF/CNPJ é obrigatório' })
    CPF_CNPJ: string;

    @IsDateString()
    DATA_CADASTRO: string;
}

```

2.7 Service de Pessoas (CRUD Completo)

```

src/pessoas/pessoas.service.ts
// src/pessoas/pessoas.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Pessoa } from '../pessoa.entity';
import { CreatePessoaDto } from '../dto/create-pessoa.dto';

@Injectable()
export class PessoasService {
  constructor(
    @InjectRepository(Pessoa)
    private repo: Repository<Pessoa>,
  ) {}

  findAll(): Promise<Pessoa[]> {
    return this.repo.find();
  }

  async findOne(id: number): Promise<Pessoa> {
    const pessoa = await this.repo.findOneBy({ ID_PESSOA: id });
    if (!pessoa)
      throw new NotFoundException(`Pessoa #${id} não encontrada`);
    return pessoa;
  }

  create(dto: CreatePessoaDto): Promise<Pessoa> {
    const nova = this.repo.create(dto);
    return this.repo.save(nova);
  }

  async update(id: number, dto: Partial<CreatePessoaDto>): Promise<Pessoa> {
    await this.findOne(id); // lança 404 se não existir
    await this.repo.update(id, dto);
    return this.findOne(id);
  }

  async remove(id: number): Promise<void> {
    await this.findOne(id);
    await this.repo.delete(id);
  }
}

```

2.8 Controller de Pessoas (CRUD Completo)

src/pessoas/pessoas.controller.ts

```
// src/pessoas/pessoas.controller.ts
import { Controller, Get, Post, Put, Delete,
        Body, Param, ParseIntPipe } from '@nestjs/common';
import { PessoasService } from '../pessoas.service';
import { CreatePessoaDto } from '../dto/create-pessoa.dto';

@Controller('pessoas')
export class PessoasController {
  constructor(private readonly service: PessoasService) {}

  @Get()
  findAll() { return this.service.findAll(); }

  @Get('/:id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.service.findOne(id);
  }

  @Post()
  create(@Body() dto: CreatePessoaDto) {
    return this.service.create(dto);
  }

  @Put('/:id')
  update(
    @Param('id', ParseIntPipe) id: number,
    @Body() dto: Partial<CreatePessoaDto>,
  ) { return this.service.update(id, dto); }

  @Delete('/:id')
  remove(@Param('id', ParseIntPipe) id: number) {
    return this.service.remove(id);
  }
}
```

2.9 Módulo de Pessoas

src/pessoas/pessoas.module.ts

```
// src/pessoas/pessoas.module.ts
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Pessoa } from '../pessoa.entity';
import { PessoasService } from '../pessoas.service';
import { PessoasController } from '../pessoas.controller';

@Module({
```

```

    imports:      [TypeOrmModule.forFeature([Pessoa])],
    providers:    [PessoasService],
    controllers:  [PessoasController],
    exports:      [PessoasService],
  })
  export class PessoasModule {}

```

 **Replicação** Repita o mesmo padrão (entity, dto, service, controller, module) para todas as outras entidades: Moradores, Unidades, Funcionarios, Fornecedores, Visitantes, Reservas, Boletos, Comunicados, Contratos, ContasPagar e ContasReceber. O padrão é sempre o mesmo — apenas mude os nomes e campos.

2.10 Registrando os Módulos em app.module.ts

Após criar todos os módulos, registre-os no AppModule:

```

src/app.module.ts final
// src/app.module.ts (versão final)
import { Module }      from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { PessoasModule } from '../pessoas/pessoas.module';
import { MoradoresModule } from '../moradores/moradores.module';
import { UnidadesModule } from '../unidades/unidades.module';
// ... importe todos os módulos criados

@Module({
  imports: [
    TypeOrmModule.forRoot({ /* config do banco */ }),
    PessoasModule,
    MoradoresModule,
    UnidadesModule,
    // ... todos os outros módulos
  ],
})
export class AppModule {}

```

2.11 Testando o Backend

Terminal — iniciar servidor

```
npm run start:dev
```

O servidor NestJS será iniciado na porta 3000. Teste no Postman:

Postman — exemplos de requisições

```

GET    http://localhost:3000/pessoas
POST   http://localhost:3000/pessoas
  Body: { "NOME": "João", "TIPO_PESSOA": "FISICA",
          "CPF_CNPJ": "12345678900", "DATA_CADASTRO": "2026-01-01" }
PUT    http://localhost:3000/pessoas/1
  Body: { "NOME": "João Atualizado" }
DELETE http://localhost:3000/pessoas/1

```

✓ **Parte 2 concluída** Backend NestJS com CRUD completo, validação de dados, tratamento de erros 404 e CORS configurado.

Parte 3 — Autenticação JWT (Implementação Detalhada)

💡 **Por que JWT?** JWT (JSON Web Token) permite que o backend verifique se o usuário está autenticado sem precisar consultar o banco de dados a cada requisição. O token é gerado no login e enviado em todas as chamadas subsequentes.

3.1 Criando a Tabela de Usuários

Adicione ao script SQL e execute no banco:

SQL — USUARIOS

```
CREATE TABLE USUARIOS (  
  ID_USUARIO INT AUTO_INCREMENT PRIMARY KEY,  
  ID_PESSOA INT,  
  LOGIN VARCHAR(50) NOT NULL UNIQUE,  
  SENHA_HASH VARCHAR(255) NOT NULL,  
  PERFIL ENUM('ADMIN', 'SINDICO', 'MORADOR') DEFAULT 'MORADOR',  
  ATIVO TINYINT(1) DEFAULT 1,  
  FOREIGN KEY (ID_PESSOA) REFERENCES PESSOAS(ID_PESSOA)  
);
```

3.2 Entidade e DTO de Usuário

src/auth/usuario.entity.ts

```
// src/auth/usuario.entity.ts  
import { Entity, PrimaryGeneratedColumn, Column } from 'typeorm';  
  
@Entity('USUARIOS')  
export class Usuario {  
  @PrimaryGeneratedColumn()  
  ID_USUARIO: number;  
  
  @Column({ nullable: true })  
  ID_PESSOA: number;  
  
  @Column({ unique: true, length: 50 })  
  LOGIN: string;  
  
  @Column({ length: 255 })  
  SENHA_HASH: string;  
  
  @Column({ default: 'MORADOR' })  
  PERFIL: string;  
  
  @Column({ default: true })  
  ATIVO: boolean;  
}
```

src/auth/dto/login.dto.ts

```
// src/auth/dto/login.dto.ts
import { IsString, IsNotEmpty } from 'class-validator';

export class LoginDto {
  @IsString() @IsNotEmpty()
  login: string;

  @IsString() @IsNotEmpty()
  password: string;
}
```

3.3 AuthService — Hash e JWT

src/auth/auth.service.ts

```
// src/auth/auth.service.ts
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { JwtService } from '@nestjs/jwt';
import * as bcrypt from 'bcryptjs';
import { Usuario } from '../usuario.entity';
import { LoginDto } from '../dto/login.dto';

@Injectable()
export class AuthService {
  constructor(
    @InjectRepository(Usuario)
    private usuarioRepo: Repository<Usuario>,
    private jwtService: JwtService,
  ) {}

  // Criar usuário com senha hash (use para cadastrar admins)
  async criarUsuario(login: string, senha: string, perfil = 'MORADOR') {
    const hash = await bcrypt.hash(senha, 12);
    const novo = this.usuarioRepo.create({
      LOGIN: login, SENHA_HASH: hash, PERFIL: perfil,
    });
    return this.usuarioRepo.save(novo);
  }

  // Login - valida credenciais e retorna JWT
  async login(dto: LoginDto): Promise<{ access_token: string }> {
    const usuario = await this.usuarioRepo.findOneBy({ LOGIN: dto.login });

    // Mensagem genérica: não revela se o login existe ou não
    if (!usuario || !(await bcrypt.compare(dto.password, usuario.SENHA_HASH)))
      throw new UnauthorizedException('Credenciais inválidas');

    if (!usuario.ATIVO)
      throw new UnauthorizedException('Usuário inativo');
```

```

    const payload = {
      sub:    usuario.ID_USUARIO,
      login:  usuario.LOGIN,
      perfil: usuario.PERFIL,
    };
    return { access_token: this.jwtService.sign(payload) };
  }
}

```

3.4 AuthController e AuthModule

src/auth/auth.controller.ts

```

// src/auth/auth.controller.ts
import { Controller, Post, Body } from '@nestjs/common';
import { AuthService } from '../auth.service';
import { LoginDto } from '../dto/login.dto';

@Controller('auth')
export class AuthController {
  constructor(private authService: AuthService) {}

  @Post('login')
  login(@Body() dto: LoginDto) {
    return this.authService.login(dto);
  }
}

```

src/auth/auth.module.ts

```

// src/auth/auth.module.ts
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { JwtModule } from '@nestjs/jwt';
import { Usuario } from '../usuario.entity';
import { AuthService } from '../auth.service';
import { AuthController } from '../auth.controller';

@Module({
  imports: [
    TypeOrmModule.forFeature([Usuario]),
    JwtModule.register({
      secret: 'SEU_SECRET_SEGURO_AQUI', // mude em produção!
      signOptions: { expiresIn: '8h' },
    }),
  ],
  providers: [AuthService],
  controllers: [AuthController],
  exports: [JwtModule],
})
export class AuthModule {}

```

⚠ **Secret JWT** O secret 'SEU_SECRET_SEGURO_AQUI' deve ser uma string longa e aleatória. Em produção, use variáveis de ambiente: `process.env.JWT_SECRET`

✓ **Parte 3 concluída** Backend com autenticação JWT completa. O endpoint `POST /auth/login` retorna um token que será usado pelo frontend.

Parte 4 — Frontend com Next.js (Refatoração e Design Dark Mode)

4.1 Criando o Projeto

Terminal — criar projeto

```
npx create-next-app@latest condominio-frontend \
  --typescript --tailwind --eslint --app --src-dir
cd condominio-frontend
```

Quando solicitado, responda:

- Would you like to use TypeScript? → Yes
- Would you like to use ESLint? → Yes
- Would you like to use Tailwind? → Yes
- Would you like to use App Router? → Yes

4.2 Instalando shadcn/ui

O shadcn/ui não é uma biblioteca comum — ele copia os componentes para dentro do seu projeto, dando controle total sobre o código.

Terminal — shadcn/ui init

```
# Inicializar shadcn/ui no projeto
npx shadcn@latest init

# Quando perguntar o tema, escolha: zinc (funciona bem com dark mode)
# Quando perguntar o diretório, mantenha o padrão: src/components/ui
```

Terminal — instalar componentes

```
# Instalar os componentes que vamos usar
npx shadcn@latest add button
npx shadcn@latest add input
npx shadcn@latest add card
npx shadcn@latest add table
npx shadcn@latest add dialog
npx shadcn@latest add form
npx shadcn@latest add select
npx shadcn@latest add badge
npx shadcn@latest add toast
npx shadcn@latest add sidebar
```

4.3 Instalando Dependências Adicionais

Terminal — dependências

```
npm install axios js-cookie
```

```
npm install --save-dev @types/js-cookie

# ícones (lucide-react já vem com shadcn)
# react-hook-form para formulários
npm install react-hook-form @hookform/resolvers zod
```

4.4 Configurando o Tema Dark Mode

Edite o arquivo `tailwind.config.ts` para garantir que o dark mode funcione:

```
tailwind.config.ts

// tailwind.config.ts
import type { Config } from 'tailwindcss'

const config: Config = {
  darkMode: ['class'],
  content: [
    './src/pages/**/*..{js,ts,jsx,tsx,mdx}',
    './src/components/**/*..{js,ts,jsx,tsx,mdx}',
    './src/app/**/*..{js,ts,jsx,tsx,mdx}',
  ],
  theme: {
    extend: {
      colors: {
        // Paleta Dark Mode roxo/índigo personalizada
        primary: {
          DEFAULT: '#7C3AED',
          foreground: 'FFFFFF',
        },
      },
    },
  },
  plugins: [require('tailwindcss-animate')],
}
export default config
```

4.5 Configurando o Layout Global Dark Mode

Edite `src/app/globals.css` para definir as variáveis de cor do tema escuro:

```
src/app/globals.css

/* src/app/globals.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --background: 222 47% 11%;
    --foreground: 210 40% 98%;
    --card: 222 47% 14%;
    --card-foreground: 210 40% 98%;
  }
}
```

```

    --primary:          263 70% 58%;
    --primary-foreground: 210 40% 98%;
    --secondary:        217 33% 18%;
    --secondary-foreground: 210 40% 98%;
    --muted:            217 33% 18%;
    --muted-foreground: 215 20% 65%;
    --accent:           263 70% 58%;
    --accent-foreground: 210 40% 98%;
    --border:           217 33% 22%;
    --input:            217 33% 18%;
    --ring:             263 70% 58%;
    --radius:           0.5rem;
  }
}

body {
  background-color: hsl(var(--background));
  color:            hsl(var(--foreground));
}

```

4.6 Serviço de API com Interceptador

Crie o arquivo `src/services/api.ts`. Este serviço adiciona automaticamente o token JWT em todas as requisições:

src/services/api.ts

```

// src/services/api.ts
import axios from 'axios';
import Cookies from 'js-cookie';

const api = axios.create({
  baseURL: 'http://localhost:3000',
  timeout: 10000,
});

// Interceptador de requisição: adiciona o token JWT automaticamente
api.interceptors.request.use((config) => {
  const token = Cookies.get('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Interceptador de resposta: redireciona para login se token expirar
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      Cookies.remove('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
)

```

```
);  
  
export default api;
```

4.7 Serviço de Autenticação

src/services/auth.ts

```
// src/services/auth.ts  
import api from './api';  
import Cookies from 'js-cookie';  
  
export async function login(login: string, password: string) {  
  const res = await api.post('/auth/login', { login, password });  
  // httpOnly não é possível no lado cliente – usamos secure cookie  
  Cookies.set('token', res.data.access_token, {  
    secure: process.env.NODE_ENV === 'production',  
    sameSite: 'strict',  
    expires: 1, // 1 dia  
  });  
  return res.data;  
}  
  
export function logout() {  
  Cookies.remove('token');  
  window.location.href = '/login';  
}  
  
export function getToken(): string | undefined {  
  return Cookies.get('token');  
}  
  
export function isAuthenticated(): boolean {  
  return !!getToken();  
}
```

✓ **Parte 4 concluída** Projeto Next.js criado com shadcn/ui, tema dark mode configurado e serviços de API prontos.

Parte 5 — Tela de Login com Dark Mode e shadcn/ui

A tela de login original era funcional mas sem estilo. Vamos refatorá-la com shadcn/ui, validação com react-hook-form e Zod, e feedback visual de erros.

5.1 Estrutura de Arquivos

- src/app/login/page.tsx → página de login
- src/app/(auth)/layout.tsx → layout para páginas autenticadas
- src/app/(auth)/dashboard/page.tsx → dashboard protegido

5.2 Tela de Login Completa

src/app/login/page.tsx

```
// src/app/login/page.tsx
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { login } from '@services/auth';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from '@components/ui/card';
import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from '@components/ui/form';

// Schema de validação com Zod
const loginSchema = z.object({
  login: z.string().min(3, 'Login deve ter ao menos 3 caracteres'),
  password: z.string().min(6, 'Senha deve ter ao menos 6 caracteres'),
});

type LoginFormData = z.infer<typeof loginSchema>;

export default function LoginPage() {
  const router = useRouter();
  const [erro, setErro] = useState('');
  const [carregando, setCarreg] = useState(false);

  const form = useForm<LoginFormData>({
    resolver: zodResolver(loginSchema),
    defaultValues: { login: '', password: '' },
  });

  const onSubmit = async (data: LoginFormData) => {
```

```

    setErro(''); setCarreg(true);
    try {
      await login(data.login, data.password);
      router.push('/dashboard');
    } catch {
      setErro('Login ou senha inválidos.');
```

```

    } finally {
      setCarreg(false);
    }
  };

```

```

return (
  <div className="min-h-screen flex items-center justify-center
    bg-background">
    <Card className="w-full max-w-md border-border bg-card shadow-2xl">
      <CardHeader className="text-center pb-2">
        <div className="mx-auto mb-4 flex h-14 w-14 items-center
          justify-center rounded-full bg-primary/20">
          <span className="text-2xl">🏠</span>
        </div>
        <CardTitle className="text-2xl font-bold text-foreground">
          Sistema de Condomínios
        </CardTitle>
        <CardDescription className="text-muted-foreground">
          Faça login para acessar o painel
        </CardDescription>
      </CardHeader>
      <CardContent>
        <Form {...form}>
          <form onSubmit={form.handleSubmit(onSubmit)}
            className="space-y-4">
            <FormField
              control={form.control} name="login"
              render={({ field }) => (
                <FormItem>
                  <FormLabel>Login</FormLabel>
                  <FormControl>
                    <Input placeholder="seu.login"
                      className="bg-input" {...field} />
                  </FormControl>
                  <FormMessage />
                </FormItem>
              )}
            />
            <FormField
              control={form.control} name="password"
              render={({ field }) => (
                <FormItem>
                  <FormLabel>Senha</FormLabel>
                  <FormControl>
                    <Input type="password" placeholder="....."
                      className="bg-input" {...field} />
                  </FormControl>
                  <FormMessage />
                </FormItem>
              )}
            />
          </form>
        </Form>
      </CardContent>
    </Card>
  </div>
);

```

```

        ))
      />
      {erro && (
        <p className="text-sm text-red-400 text-center">
          {erro}
        </p>
      )}
      <Button type="submit" className="w-full bg-primary
        hover:bg-primary/90" disabled={carregando}>
        {carregando ? 'Entrando...' : 'Entrar'}
      </Button>
    </form>
  </Form>
</CardContent>
</Card>
</div>
);
}

```

☑ **Parte 5 concluída** Tela de login dark mode com validação Zod, feedback de erro e carregamento. Muito mais profissional que a versão original.

Parte 6 — Dashboard Moderno com Sidebar e Cards de Métricas

6.1 Layout com Sidebar

Crie o layout base que envolve todas as páginas autenticadas:

src/app/(auth)/layout.tsx

```
// src/app/(auth)/layout.tsx
'use client';

import { useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { isAuthenticated, logout } from '@services/auth';
import { Button } from '@components/ui/button';
import Link from 'next/link';
import {
  Home, Users, Building2, Wrench, Truck, UserCheck,
  Calendar, FileText, Receipt, LogOut, Settings
} from 'lucide-react';

const navItems = [
  { href: '/dashboard', label: 'Painel', icon: Home },
  { href: '/pessoas', label: 'Pessoas', icon: Users },
  { href: '/moradores', label: 'Moradores', icon: Building2 },
  { href: '/funcionarios', label: 'Funcionários', icon: Wrench },
  { href: '/fornecedores', label: 'Fornecedores', icon: Truck },
  { href: '/visitantes', label: 'Visitantes', icon: UserCheck },
  { href: '/unidades', label: 'Unidades', icon: Building2 },
  { href: '/areas-comuns', label: 'Áreas Comuns', icon: Calendar },
  { href: '/reservas', label: 'Reservas', icon: Calendar },
  { href: '/boletos', label: 'Boletos', icon: Receipt },
  { href: '/comunicados', label: 'Comunicados', icon: FileText },
  { href: '/contratos', label: 'Contratos', icon: FileText },
  { href: '/contas-pagar', label: 'Contas Pagar', icon: Receipt },
  { href: '/contas-receber', label: 'Contas Receber', icon: Receipt },
];

export default function AuthLayout({
  children,
}: { children: React.ReactNode }) {
  const router = useRouter();

  useEffect(() => {
    if (!isAuthenticated()) router.push('/login');
  }, [router]);

  return (
    <div className="flex h-screen bg-background">
      {/* Sidebar */}
    </div>
  );
}
```

```

<aside className="w-64 border-r border-border bg-card flex flex-col">
  <div className="p-6 border-b border-border">
    <h1 className="text-lg font-bold text-foreground">
      🏠 Condomínio
    </h1>
    <p className="text-xs text-muted-foreground mt-1">
      Sistema de Gestão
    </p>
  </div>
  <nav className="flex-1 overflow-y-auto p-3 space-y-1">
    {navItems.map(({ href, label, icon: Icon }) => (
      <Link key={href} href={href}
        className="flex items-center gap-3 px-3 py-2 rounded-md
          text-sm text-muted-foreground
          hover:text-foreground hover:bg-secondary
          transition-colors">
        <Icon className="h-4 w-4" />
        {label}
      </Link>
    ))}
  </nav>
  <div className="p-3 border-t border-border">
    <Button variant="ghost" className="w-full justify-start gap-3
      text-muted-foreground hover:text-foreground"
      onClick={logout}>
      <LogOut className="h-4 w-4" /> Sair
    </Button>
  </div>
</aside>
{/* Conteúdo principal */}
<main className="flex-1 overflow-y-auto p-6">
  {children}
</main>
</div>
);
}

```

6.2 Dashboard com Cards de Métricas

src/app/(auth)/dashboard/page.tsx

```

// src/app/(auth)/dashboard/page.tsx
'use client';

import { useEffect, useState } from 'react';
import api from '@/services/api';
import { Card, CardContent, CardHeader, CardTitle }
  from '@/components/ui/card';
import { Badge } from '@/components/ui/badge';
import { Users, Building2, Receipt, Calendar } from 'lucide-react';

export default function DashboardPage() {
  const [stats, setStats] = useState({
    pessoas: 0, moradores: 0, unidades: 0,

```

```

    boletosAbertos: 0, reservasHoje: 0,
  });

useEffect(() => {
  async function carregar() {
    try {
      const [pess, mor, unid, bol] = await Promise.all([
        api.get('/pessoas'),
        api.get('/moradores'),
        api.get('/unidades'),
        api.get('/boletos'),
      ]);
      setStats({
        pessoas:      pess.data.length,
        moradores:    mor.data.length,
        unidades:      unid.data.length,
        boletosAbertos: bol.data.filter(
          (b: any) => b.STATUS === 'ABERTO').length,
        reservasHoje: 0,
      });
    } catch (err) {
      console.error('Erro ao carregar estatísticas', err);
    }
  }
  carregar();
}, []);

const cards = [
  { title: 'Pessoas', value: stats.pessoas,
    icon: Users, color: 'text-violet-400' },
  { title: 'Moradores', value: stats.moradores,
    icon: Users, color: 'text-indigo-400' },
  { title: 'Unidades', value: stats.unidades,
    icon: Building2, color: 'text-blue-400' },
  { title: 'Boletos Abertos', value: stats.boletosAbertos,
    icon: Receipt, color: 'text-amber-400' },
];

return (
  <div className="space-y-6">
    <div>
      <h1 className="text-2xl font-bold text-foreground">
        Painel do Condomínio
      </h1>
      <p className="text-muted-foreground mt-1">
        Visão geral do sistema
      </p>
    </div>
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
      {cards.map(({ title, value, icon: Icon, color }) => (
        <Card key={title} className="bg-card border-border">
          <CardHeader className="flex flex-row items-center
            justify-between pb-2">
            <CardTitle className="text-sm font-medium

```

```

        text-muted-foreground">{title}</CardTitle>
        <Icon className={`h-5 w-5 ${color}`} />
      </CardHeader>
      <CardContent>
        <p className="text-3xl font-bold text-foreground">
          {value}
        </p>
      </CardContent>
    </Card>
  )}
</div>
</div>
);
}

```

☒ **Parte 6 concluída** Dashboard com sidebar de navegação, cards de métricas e carregamento de dados reais do backend.

Parte 7 — Páginas CRUD Refatoradas (Tabelas e Formulários)

O CRUD original usava inputs sem estilo e listas simples. Vamos refatorar para usar tabelas, diálogos modais e feedback visual com toasts — tornando o sistema muito mais profissional.

7.1 Padrão de Página CRUD

Todas as páginas CRUD seguirão este padrão:

- Tabela com shadcn/ui para listagem dos registros
- Botão 'Novo' que abre um Dialog (modal) para cadastro
- Botão 'Editar' que abre o mesmo Dialog com os dados preenchidos
- Botão 'Excluir' com confirmação via AlertDialog
- Toast de sucesso ou erro após cada operação

7.2 Página de Pessoas — CRUD Completo Refatorado

src/app/(auth)/pessoas/page.tsx — CRUD refatorado completo

```
// src/app/(auth)/pessoas/page.tsx
'use client';

import { useState, useEffect, useCallback } from 'react';
import api from '@services/api';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Badge } from '@components/ui/badge';
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from '@components/ui/table';
import {
  Dialog, DialogContent, DialogHeader,
  DialogTitle, DialogFooter,
} from '@components/ui/dialog';
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from '@components/ui/alert-dialog';
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from '@components/ui/select';
import { useToast } from '@components/ui/use-toast';
import { Plus, Pencil, Trash2 } from 'lucide-react';

type Pessoa = {
```

```

    ID_PESSOA: number;
    NOME: string;
    TIPO_PESSOA: 'FISICA' | 'JURIDICA';
    CPF_CNPJ: string;
    DATA_CADASTRO: string;
  };

  const FORM_VAZIO = {
    NOME: '', TIPO_PESSOA: '', CPF_CNPJ: '', DATA_CADASTRO: '',
  };

  export default function PessoasPage() {
    const { toast } = useToast();
    const [pessoas, setPessoas] = useState<Pessoa[]>([]);
    const [form, setForm] = useState(FORM_VAZIO);
    const [editId, setEditId] = useState<number | null>(null);
    const [dialogOpen, setDialog] = useState(false);
    const [excluirId, setExcluir] = useState<number | null>(null);
    const [carregando, setLoad] = useState(false);

    const carregar = useCallback(async () => {
      const res = await api.get('/pessoas');
      setPessoas(res.data);
    }, []);

    useEffect(() => { carregar(); }, [carregar]);

    const abrirNovo = () => {
      setForm(FORM_VAZIO); setEditId(null); setDialog(true);
    };

    const abrirEditar = (p: Pessoa) => {
      setForm({
        NOME: p.NOME,
        TIPO_PESSOA: p.TIPO_PESSOA,
        CPF_CNPJ: p.CPF_CNPJ,
        DATA_CADASTRO: p.DATA_CADASTRO,
      });
      setEditId(p.ID_PESSOA);
      setDialog(true);
    };

    const salvar = async () => {
      setLoad(true);
      try {
        if (editId) await api.put(`/pessoas/${editId}`, form);
        else await api.post('/pessoas', form);
        toast({ title: editId ? 'Pessoa atualizada!' : 'Pessoa cadastrada!',
          description: form.NOME });
        setDialog(false);
        carregar();
      } catch {
        toast({ title: 'Erro', description: 'Não foi possível salvar.'},

```

```

        variant: 'destructive' });
    } finally { setLoad(false); }
};

const confirmarExclusao = async () => {
  if (!excluirId) return;
  try {
    await api.delete(`/pessoas/${excluirId}`);
    toast({ title: 'Pessoa excluída com sucesso.' });
    setExcluir(null);
    carregar();
  } catch {
    toast({ title: 'Erro ao excluir', variant: 'destructive' });
  }
};

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h1 className="text-xl font-bold text-foreground">Pessoas</h1>
      <Button onClick={abrirNovo}
        className="bg-primary hover:bg-primary/90">
        <Plus className="h-4 w-4 mr-2" /> Nova Pessoa
      </Button>
    </div>

    <div className="rounded-md border border-border">
      <Table>
        <TableHeader>
          <TableRow className="border-border hover:bg-secondary/50">
            <TableHead className="text-muted-foreground">#</TableHead>
            <TableHead className="text-muted-foreground">Nome</TableHead>
            <TableHead className="text-muted-foreground">Tipo</TableHead>
            <TableHead className="text-muted-foreground">CPF/CNPJ</TableHead>
            <TableHead className="text-muted-foreground">Cadastro</TableHead>
            <TableHead className="text-muted-foreground text-right">
              Ações
            </TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          {pessoas.map((p) => (
            <TableRow key={p.ID_PESSOA}
              className="border-border hover:bg-secondary/30">
              <TableCell className="text-muted-foreground">
                {p.ID_PESSOA}
              </TableCell>
              <TableCell className="font-medium text-foreground">
                {p.NOME}
              </TableCell>
              <TableCell>
                <Badge variant={p.TIPO_PESSOA === 'FISICA'
                  ? 'secondary' : 'outline'}>
                  {p.TIPO_PESSOA}
                </Badge>
              </TableCell>
            </TableRow>
          ))}
        </TableBody>
      </Table>
    </div>
  </div>
);

```

```

        </Badge>
      </TableCell>
      <TableCell className="text-muted-foreground">
        {p.CPF_CNPJ}
      </TableCell>
      <TableCell className="text-muted-foreground">
        {new Date(p.DATA_CADASTRO).toLocaleDateString('pt-BR')}
      </TableCell>
      <TableCell className="text-right">
        <div className="flex justify-end gap-2">
          <Button variant="ghost" size="icon"
            onClick={() => abrirEditar(p)}>
            <Pencil className="h-4 w-4 text-violet-400" />
          </Button>
          <Button variant="ghost" size="icon"
            onClick={() => setExcluir(p.ID_PESSOA)}>
            <Trash2 className="h-4 w-4 text-red-400" />
          </Button>
        </div>
      </TableCell>
    </TableRow>
  </TableBody>
</Table>
</div>

{/* Dialog de cadastro/edição */}
<Dialog open={dialogOpen} onOpenChange={setDialog}>
  <DialogContent className="bg-card border-border">
    <DialogHeader>
      <DialogTitle>
        {editId ? 'Editar Pessoa' : 'Nova Pessoa'}
      </DialogTitle>
    </DialogHeader>
    <div className="space-y-3 py-2">
      <Input placeholder="Nome completo" value={form.NOME}
        className="bg-input"
        onChange={e => setForm({ ...form, NOME: e.target.value })} />
      <Select value={form.TIPO_PESSOA}
        onChange={v => setForm({ ...form, TIPO_PESSOA: v })}>
        <SelectTrigger className="bg-input">
          <SelectValue placeholder="Tipo de pessoa" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="FISICA">Física</SelectItem>
          <SelectItem value="JURIDICA">Jurídica</SelectItem>
        </SelectContent>
      </Select>
      <Input placeholder="CPF ou CNPJ" value={form.CPF_CNPJ}
        className="bg-input"
        onChange={e => setForm({ ...form, CPF_CNPJ: e.target.value })} />
      <Input type="date" value={form.DATA_CADASTRO}
        className="bg-input"
        onChange={e => setForm({ ...form, DATA_CADASTRO:
e.target.value })} />

```

```

        </div>
        <DialogFooter>
          <Button variant="outline" onClick={() => setDialog(false)}>
            Cancelar
          </Button>
          <Button className="bg-primary hover:bg-primary/90"
            onClick={salvar} disabled={carregando}>
            {carregando ? 'Salvando...' : 'Salvar'}
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>

    { /* AlertDialog de confirmação de exclusão */ }
    <AlertDialog open={!excluirId}
      onOpenChange={() => setExcluir(null)}>
      <AlertDialogContent className="bg-card border-border">
        <AlertDialogHeader>
          <AlertDialogTitle>Confirmar exclusão</AlertDialogTitle>
          <AlertDialogDescription>
            Esta ação não pode ser desfeita.
          </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel>Cancelar</AlertDialogCancel>
          <AlertDialogAction
            className="bg-red-600 hover:bg-red-700"
            onClick={confirmarExclusao}>
            Excluir
          </AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>
  </div>
);
}

```

 **Replicando para outras entidades** Siga exatamente o mesmo padrão para todas as demais entidades (Moradores, Unidades, Boletos, etc.). Mude apenas: o nome do tipo TypeScript, a URL da API, os campos do formulário e as colunas da tabela.

7.3 Adicionando o Toaster no Layout

Para os toasts aparecerem, adicione o componente Toaster ao layout raiz:

```

src/app/layout.tsx
// src/app/layout.tsx - adicionar Toaster
import { Toaster } from '@components/ui/toaster';

export default function RootLayout({ children }) {
  return (
    <html lang="pt-BR">
      <body>

```

```
        {children}
        <Toaster />    {/* toast de feedback */}
    </body>
</html>
);
}
```

☑ **Parte 7 concluída** CRUD profissional com tabela estilizada, modal de cadastro/edição, confirmação de exclusão e feedback visual via toast.

Parte 8 — Integração Final, Testes e Checklist

8.1 Fluxo Completo do Sistema

Com todas as partes implementadas, o fluxo completo do sistema funciona assim:

1. Usuário acessa `http://localhost:3001/login`
2. Preenche login e senha → frontend chama `POST /auth/login` no backend
3. Backend valida as credenciais, gera um JWT e retorna ao frontend
4. Frontend salva o token em cookie seguro e redireciona para `/dashboard`
5. O interceptador do Axios adiciona o token em toda requisição subsequente
6. Se o token expirar, o interceptador redireciona automaticamente para `/login`

8.2 Iniciando o Sistema Completo

Terminal — iniciar frontend e backend

```
# Terminal 1 — Backend
cd condominio-backend
npm run start:dev
# Rodando em http://localhost:3000

# Terminal 2 — Frontend
cd condominio-frontend
npm run dev
# Rodando em http://localhost:3001
```

⚠ **Porta do Frontend** O Next.js sobe na porta 3000 por padrão, mas o backend já usa essa porta. Use `'npm run dev -- --port 3001'` para o frontend, ou configure a variável `PORT=3001` no `package.json`.

8.3 Testando com Postman — Fluxo Completo

Passo 1 — Criar um usuário admin (apenas uma vez):

Postman — criar usuário

```
POST http://localhost:3000/auth/criar-usuario
Body: {
  "login": "admin",
  "password": "senha123",
  "perfil": "ADMIN"
}
```

Passo 2 — Fazer login e pegar o token:

Postman — login

```
POST http://localhost:3000/auth/login
Body: { "login": "admin", "password": "senha123" }
```

```
Resposta:
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Passo 3 — Usar o token nas próximas requisições:

Postman — requisição autenticada

GET http://localhost:3000/pessoas

Header: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

8.4 Checklist Final

Antes de considerar o projeto concluído, verifique cada item:

	Item	Status
☐	Banco DBCONDOMINIO criado com todas as tabelas	Backend
☐	Backend NestJS rodando na porta 3000	Backend
☐	CRUD completo (GET/POST/PUT/DELETE) para todas as entidades	Backend
☐	Validação com class-validator nos DTOs	Backend
☐	Autenticação JWT implementada	Backend
☐	Senha salva com bcrypt (nunca em texto puro)	Backend
☐	CORS configurado para aceitar o frontend	Backend
☐	Frontend Next.js rodando na porta 3001	Frontend
☐	shadcn/ui instalado e tema dark mode configurado	Frontend
☐	Tela de login com validação Zod funcionando	Frontend
☐	Sidebar de navegação implementada	Frontend
☐	Dashboard com cards de métricas reais	Frontend
☐	CRUD de Pessoas refatorado com tabela e modal	Frontend
☐	Toast de feedback em todas as	Frontend

	operações	
☐	Redirecionamento para /login quando token expira	Frontend
☐	Proteção de rotas no layout (isAuthenticated)	Frontend

8.5 Próximos Passos Sugeridos

- Variáveis de ambiente: usar arquivo .env para guardar senha do banco, secret JWT e URL da API
- Paginação: adicionar paginação nas listagens para não carregar todos os registros de uma vez
- Busca e filtros: adicionar campo de busca nas tabelas de listagem
- Perfis de acesso: usar o campo PERFIL do usuário para exibir apenas os menus permitidos
- Deploy: publicar o backend no Railway ou Render, e o frontend na Vercel

☒ **Projeto Concluído!** Sistema Full Stack de Gestão de Condomínios implementado com banco MySQL, backend NestJS com JWT, e frontend Next.js com design Dark Mode usando shadcn/ui.